

Cumhuriyet Üniversitesi

İşletim Sistemleri Dersi

Bölüm 2

İşletim Sistemleri Yapısı

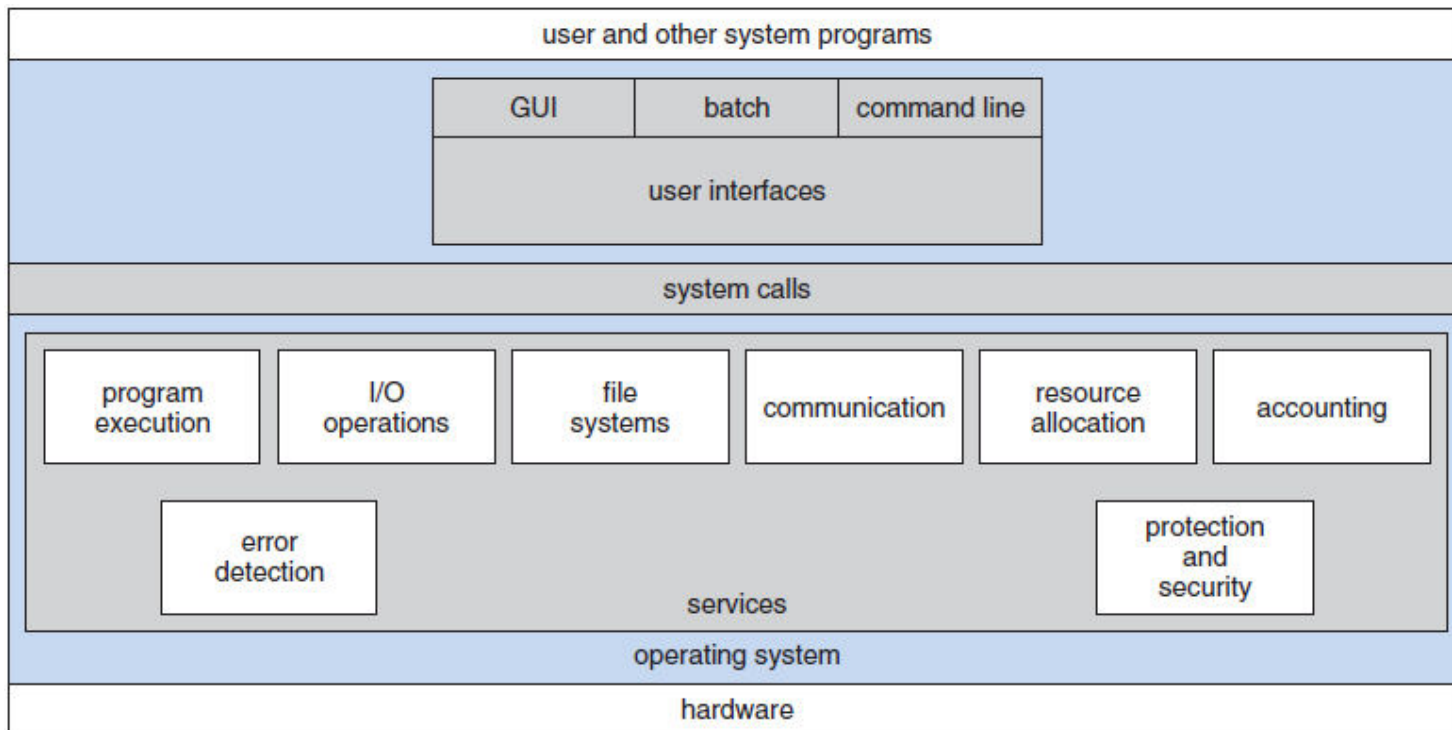
Dr. Halil ARSLAN

Bölümün hedefleri

- İşletim sistemi servisleri
- Kullanıcı-işletim sistemi arayüzleri
- Sistem çağrıları
- Sistem çağrı tipleri
- Sistem programlama
- İşletim sistemi yapısı
- İşletim sistemi hata ayıklama
- Kernel modül geliştirme

İşletim sistemi servisleri

- İşletim sistemleri programların yürütülmesi için ortam sağlar. Programlara ve kullanıcılara belirli hizmetler sunmaktadır.



İşletim sistemi servisleri

- **User interface:** İşletim sistemlerinin çoğu komut satırı (**CLI**), toplu iş dosyaları (**batch**) veya grafiksel arayüz (**GUI**) şeklinde bir kullanıcı arayüzüne (**UI**) sahiptir.
- **Program execution:** Bir programı çalıştırmak için ana belleğe yüklemeli, çalıştırmalı ve normal/anormal (hatayı raporlayarak) sonlandırmalıdır.
- **I/O operations:** Çalışan programlar, aygıt veya dosya gibi bir G/Ç gerektirebilir. Kullanıcılar doğrudan G/Ç aygıtlarını verimlilik ve güvenlik açısından kontrol edemedikleri için işletim sistemleri G/Ç aygıtlarını yönetmek için araçlar sağlamalıdır.
- **File-system:** Ana bellekte yürütülecek programlar, dosya ve dizinlere erişmek, okumak, yazmak, değiştirmek, aramak, silmek vb. manipülasyonları belli izin ve yetkilendirme kapsamında yapmaları gerekir.
- **Communications:** İşletim sistemi process'ler arasında data alışverişi gereksinimlerini sağlamalıdır. Bu süreç; bir soket, paylaşılan bellek, dosya vb. yapılardan (IPC) sağlanabilir.

İşletim sistemi servisleri

- **Error detection:** İşletim sistemi hataları, tespit etmeli ve sürekli düzeltmelidir. Bu hatalar, genellikle CPU ve ana bellek üzerinde veya G/Ç aygıtlarında, aritmetik taşma, yetkisiz hafıza erişimi, diske yazma hatası gibi donanımlarda meydana gelebilir.
- **Resource allocation:** Aynı anda birden fazla çalışan kullanıcı veya iş olduğunda kaynaklar bunlar arasında paylaştırılmalıdır.
- **Accounting:** Kullanıcıların hangi bilgisayar kaynaklarını ne kadar kullanacaklarının takip edilmesi gerekir. Bu raporlar, sistemin takibi ve iyileştirilmesi için kullanılabilir.
- **Protection and security:** Çok kullanıcılı veya ağa bağlı sistemlerdeki bilgilerin erişimi kontrollü olmalıdır. Birkaç iş birlikte çalıştırıldığında işlerden biri, diğerlerine yada işletim sistemine müdahale edememelidir. Kimlik doğrulama, erişim kısıtları ve kullanıcı/veri sahiplik ilişkileri bu güvenlik adımlarının parçalarıdır.

Kullanıcı-İşletim sistemi arayüzleri

İşletim sistemleri kullanıcılarla komut yorumlayıcı (**command interpreter**) yada grafiksek arayüzler (**GUI**) aracılığıyla iletişim kurarlar.

Komut Yorumlayıcı:

- Komut yorumlayıcıların ana işlevi kullanıcıdan komutları almak, çalıştırmak ve sonuçlarını kullanıcıya dönmektir.
- Komut yorumlayıcıları sistem başlatıldığında aktif olabileceği gibi Unix ve Windows sistemlerinde olduğu gibi sonradan bir uygulama olarak da başlatılabilir.
- Komut yorumlayıcıları komutların kodlarını kendileri yürütebilecekleri gibi başka program kodlarını da yürütebilirler
- Bir sistemde farklı özellikler içeren birden fazla komut yorumlayıcı olabilir.

Bu komutlardan `rm` bir başka komut dosyası iken `del` komut yorumlayıcısının bir fonksiyonudur.

--> *rm -rf dosya.txt, ...> del dosya.txt*

Kullanıcı-İşletim sistemi arayüzleri

Grafik Arayüzü:

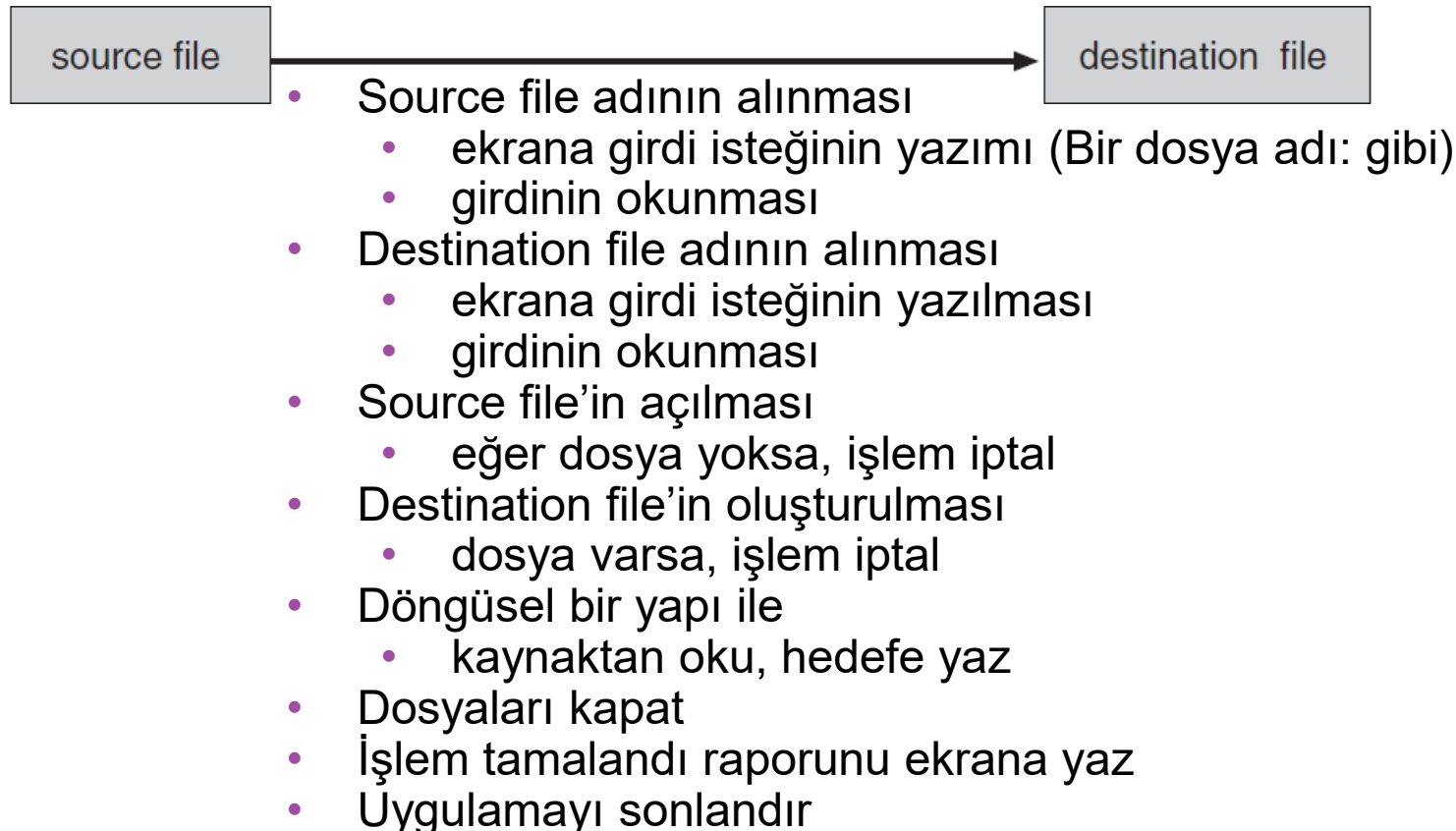
- Kullanıcı dostu grafik arayüzleri sunan stratejidir
- İlk kullanıcı arayüzü 1970'ler Xerox PARC araştırma biriminde ortaya çıksa da 1980'lerde Apple Macintosh ile yaygınlaştı.
- Unix ve Linux sistemler için IBM AIX, KDE ve GNOME grafik ekranları bu süreci takip etmiştir.

Arayüz Seçimi

Bir komut yorumlayıcısı yada gui seçimi çoğunlukla kullanıcının tercihi bırakılmışken, sistem yöneticileri çoğunlukla, süreçlere daha hızlı erişim, fonksiyonların özelleştirilmiş parametrik yapılarını kullanabilme, sık tekrar eden işlemleri yapabilme ve toplu iş dosyaları tanımlayabilme adına komut yorumlayıcılarını tercih ederler.

Sistem çağrıları

Sistem çağrıları, hazır servisler için işletim sistemleri tarafından bir arayüz sunar. Bu çağrılar (donanıma doğrudan erişmek gibi) C veya C++ gibi rutinleri olabileceği gibi doğrudan assembly komut setinde de kullanılabilir. Örneğin, bir dosyanın okunduğu ve içeriğin diğer bir dosyaya yazıldığı senaryoda sistem çağrıları;



Sistem çağrıları

Basit programlarda bile işletim sistemi yoğunlukla kullanılır. Ancak bunları genellikle programcı hissetmez. Uygulama geliştiriciler sistem çağrılarına **Application Programming Interface (API)**'ler üzerinden erişirler.

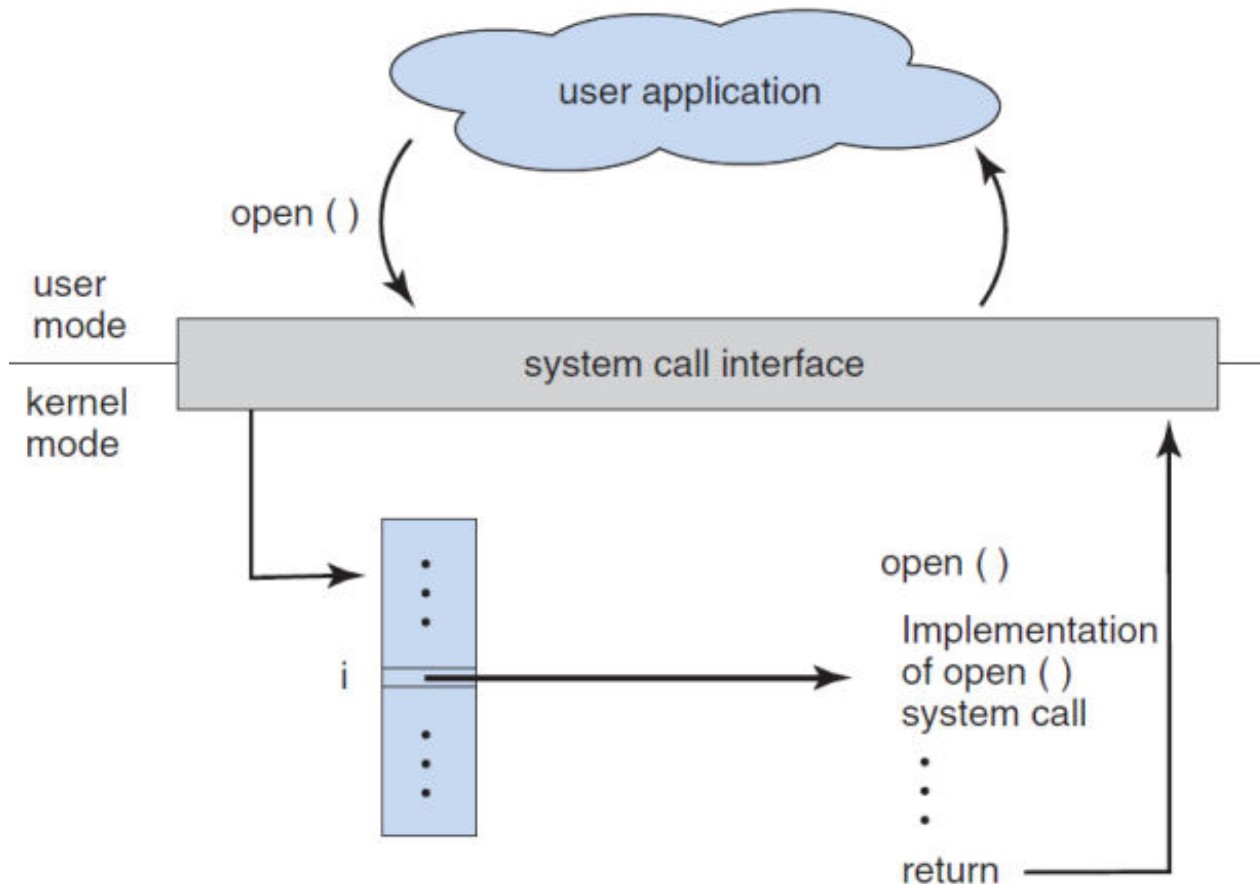
En çok bilinen API'ler Windows Sistem API'leri, POSIX sistemler (unix, linux, OsX) için POSIX API'leri ve Java Sanal Makinelerinde çalışan Java API'leridir.

Doğrudan sistem çağrıları yerine API fonksiyonlarının kullanım amacı;

- Programların taşınabilirliğini artırmak,
- Kullanım kolaylığı (sistem çağrıları daha karmaşık)
- API fonksiyonları genellikle birden fazla sistem çağrısıyla ilişkili olabilir.

Sistem çağrıları

Çoğu programlama dili, çalışma zamanında, işletim sistemi sistem çağrıları için hizmet veren bir sistem çağrısı arabirimi sağlar. Bu arayüz yakaladığı API fonksiyonuna karşılık gelen sistem çağrısını çağırır.

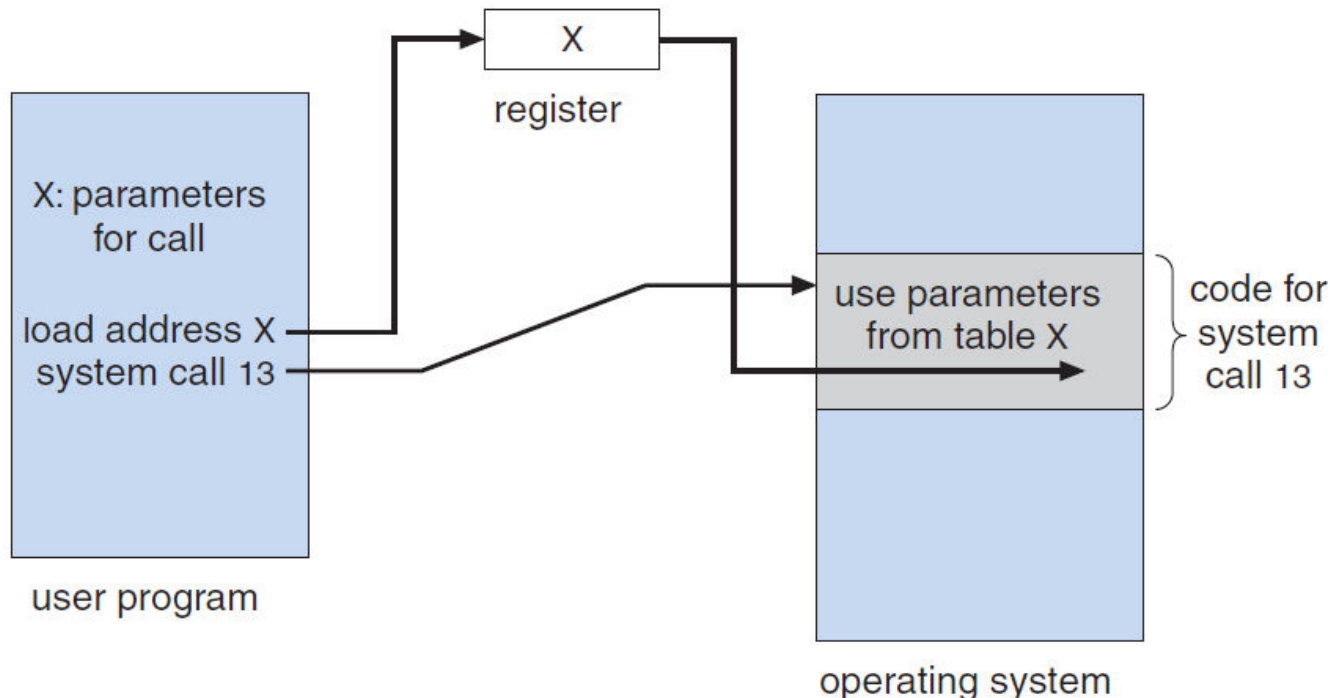


Sistem çağrıları

Sistem çağrılarına parametre geçmek için genellikle 3 yöntem kullanılır.

- Registers (kaydediciler) üzerinden. (Fazla parametre varsa!)
- Bir bellek bloğu üzerinden parametre geçmek
- Stack üzerinden parametre geçmek.

Çoğu işletim sistemi son iki metodu kullanır.



Bellek bloğundan parametre geçimi.

Sistem çağrı tipleri

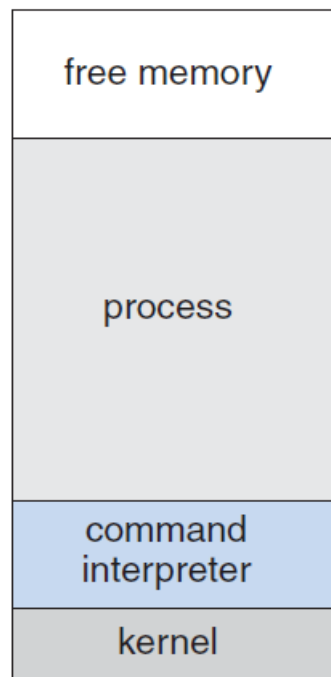
Sistem çağrıları kabaca iş yönetimi, dosya yönetimi, aygıt yönetimi, bilgi hizmetleri, iletişim ve koruma şeklinde 6 grupta toplanabilir.

1. İş yönetimi:

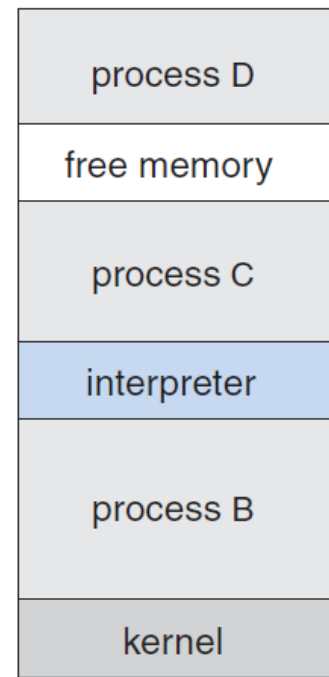
- Çalışan programların yürütümünün normal (`end()`) yada anormal (`abort()`) durdurulması,
- Bir programın çalıştırılması (`load()` veya `execute()`),
- Bir program yürütülüyorken yeni bir program çağrımı (`create_process()` veya `submit_job()`),
- Yürütülen işlerin iş önceliği, izin verilen maksimum CPU zamanı gibi parametrelerinin öğrenilmesi ve değiştirilmesi (`get/set_process_attributes()`),
- Bir işin istenildiği zaman sonlandırılması (`terminate_process()`),
- Belli bir süre işi bekletmek (`wait_time()`), özel bir olay gerçekleşinceye kadar beklemek (`wait_event()`) veya bu olayın meydana geldiğinin bildirimi (`signal_event()`),
- Bir işin kullandığı veri kaynaklarını kilitlemek (`acquire_lock()`) veya serbest bırakmak (`release_lock()`) iş yönetiminin parçalarıdır.

Sistem çağrı tipleri

1. İş yönetimi (devam): İş yönetiminin, tek görevli (**singletasking**) ve çok görevli (**multitasking**) sistemler olarak varyasyonları vardır. MS-DOS tek görevli, FreeBSD çok görevli sistemlere örnek olarak gösterilebilir.



Singletasking



Multitasking

Sistem çağrı tipleri

2. Dosya yönetimi: Dosya yönetimi ayrı bir bölüm olarak ele alınsa da ortak sistem çağrıları açısından;

- Dosya oluşturmak ve silmek (**create()** ve **delete()**),
- Dosyaya erişmek ve kullanabilmek için açmak (**open()**),
- Dosyayı okumak, yazmak, konumlanmak (**read()**, **write()**, **reposition()**)
- Dosyayı kapatmak (**close()**)
- Dosya özniteliklerini işlemek (**get/set_file_attributes()**)
- Dosya taşıma, kopyalamak vb. manipülasyonlar (**move()**, **copy()**, ...)

3. Aygıt yönetimi: İşler disk, bellek, dosya gibi kaynaklara ihtiyaç duyduklarında,

- Bir aygıta erişim isteğini iletmek (**request()**, dosya işlemlerinde **open()**),
- Erişilen aygıtı serbest bırakmak (**release()**, dosya işlemlerinde **close()**),
- Dosya yönetimine benzer şekilde (**read()**, **write()**, vb..) aygıt yönetimi pek çok işletim sisteminde kombine sistem çağrıları ile sürdürülmektedir.

Sistem çağrı tipleri

4. Bilgi hizmetleri: Çoğu işletim sistemi sistem bilgileri için çağrılar bulundurur;

- Sistem tarihi ve zamanını öğrenmek (**time()** ve **date()**),
- Sistemle ilgili mevcut kullanıcı sayısı, işletim sistemi versiyonu, bellek ve disk alanı sorgulamalar,
- Program hata ayıklamaları için bellek dökümü (**dump()**),
- Programların adım adım çalıştırılması görmek (**trap mode**)
- Yürütülen işlem bilgilerine erişmek (**get/set_process_attributes()**)

5. İletişim: İşlemler arasında iletişim için yaygın olarak kullanılan iki model vardır. Bunlar **message-passing** ve **shared-memory**'dir.

• **Message-passing:** Birbirlerine mesajların aktarımı için bir ağ üzerinden bağlantının açılması ve diğer iletişim süreçlerini içerir.

- `get_hostid()`, `get_processid()`, `open_connection()`, `close_connection()`, `accept_connection()`, `write_message()`, `read_message()`, vb.

• **Shared-memory:** Normalde işletim sistemi bir iş'in kullandığı belleğe diğer bir işin erişmesine izin vermez. Bu modelde buna izin verilir.

`shared_memory_create()`, `shared_memory_attach()`

Sistem çağrı tipleri

6. Koruma: Bir bilgisayar sisteminin kaynaklarına erişimi kontrol etmek için kullandığı mekanizmadır. Tipik olarak koruma altında toplanabilecek sistem çağrıları

- Kaynakların yetkilerini işleyen (**get/set_permission()**),
- İlgili kullanıcıyı yetkilendirmek/yetkisini almak (**allow_user()**, **deny_user()**)

Sistem programları

Sistem araçları olarak da bilinen sistem programları, program geliştirme ve yürütme için uygun ortam sağlar. Bazıları sadece sistem çağrıları için arayüz de sağlayabilir.

- **Dosya yönetimi:** create, delete, copy, rename vb. araçlar
- **Durum bilgisi:** date, time, disk, cpu ve bellek durumu vb. araçlar
- **Dosya değişiklikleri:** Dosyaları işleyen metin editörleri vb. araçlar
- **Programlama dili desteği:** Derleyiciler, hata ayıklayıcılar, entegratörler ve yorumlayıcılar gibi (C,C++,Perl, Java) araçlar
- **Program yükleme ve çalıştırma:** Hem yüksek seviyeli diller hem de makine dilleri için hata ayıklama sistemleri, yükleyiciler vb. araçlar
- **İletişim araçları:** İşler, kullanıcılar ve bilgisayarlar arasında sanal bağlantılar oluşturan araçlar (tarayıcılar, e-posta, dosya transferi vb.)
- **Arkaplan servisleri:** Belirli bir işlemi yerine getirdikten sonra sonlanan işlere görev denilirken, sürekli çalışan sistem programlarına *service*, *subsystems* veya *daemons* adı verilir. Ağ servisi böyle bir iştir.

İşletim sistemi yapısı

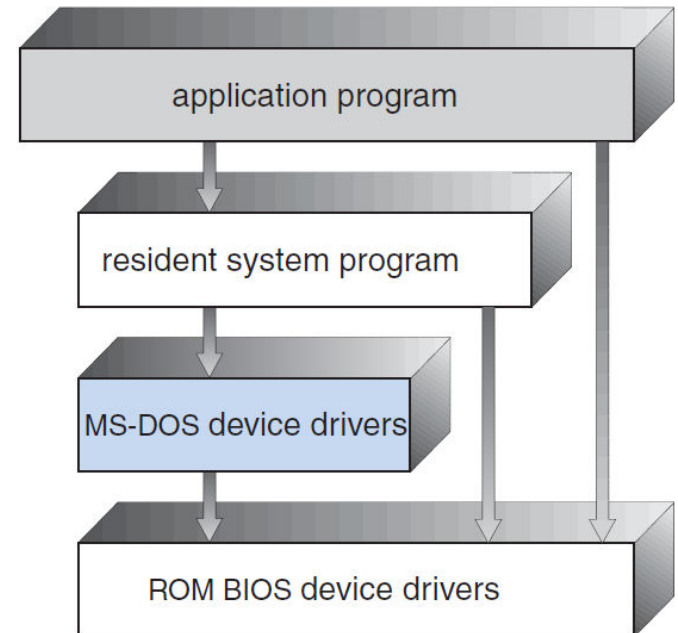
Modern bir işletim sistemi, bakım kolaylığı ve sorunsuz çalışması için;

- Görevlerin ve küçük parçaların, modüllerin içerisinde tanımlandığı bir yapıya,
- Giriş, çıkış ve fonksiyonları ile çok iyi tanımlanmış ve birbirlerinden soyutlanmış bir yapıya sahip olmalıdır.

Basit Mimari(Monolitik – Tek Parça)

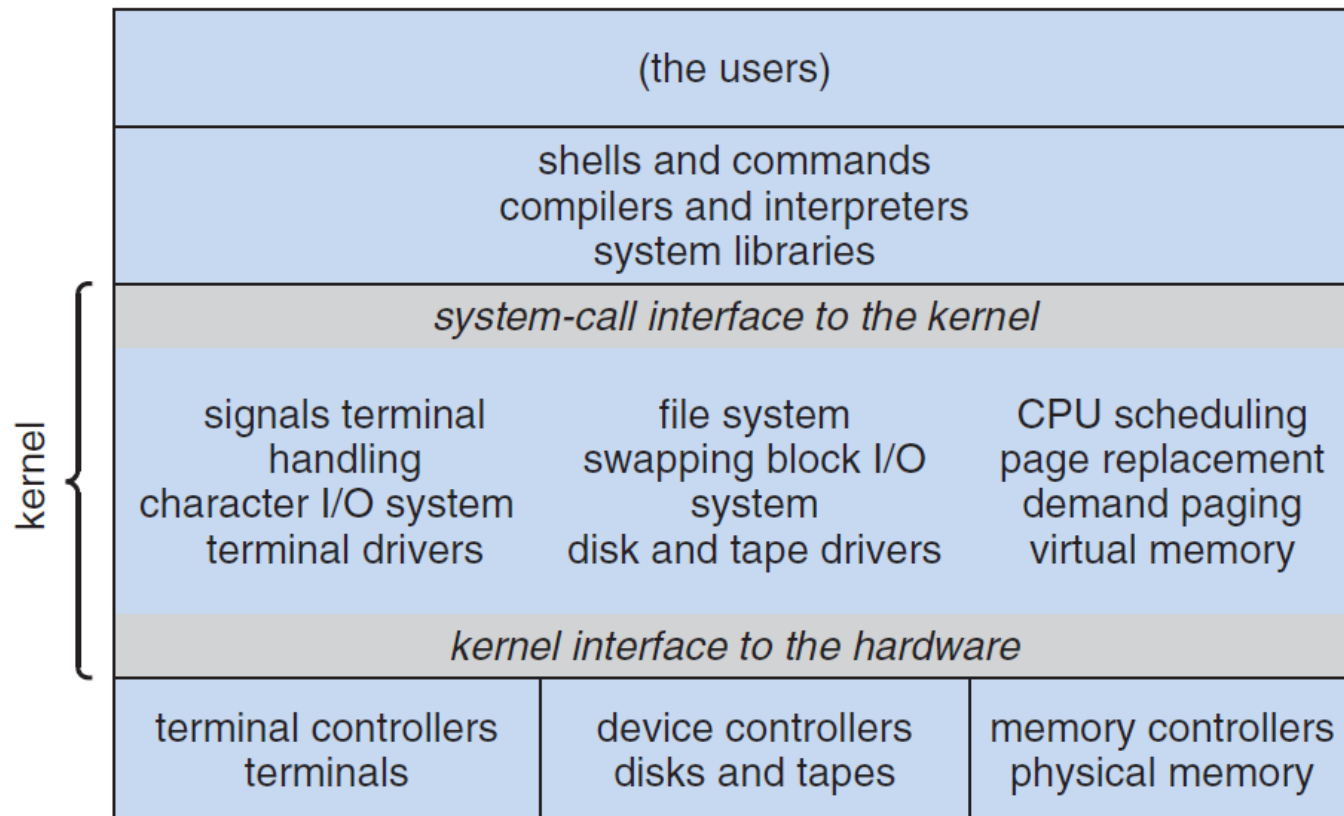
Başlangıçta MS-DOS işletim sistemi çok kapsamlı bir modüler yapıda geliştirilmemiştir.

Örneğin MS-DOS işletim sisteminde uygulama programları temel I/O rutinlerine ve disk sürücülerine doğrudan erişebilir.



İşletim sistemi yapısı - Basit Mimari

MS-DOS gibi başlangıçta UNIX işletim sistemi de donanım işlevselliği ile sınırlı kalmıştır. Ancak bu basit yapıları, sistem çağrıları ve çekirdek arayüzlerine erişim için iyi bir performans avantajı getirmektedir.

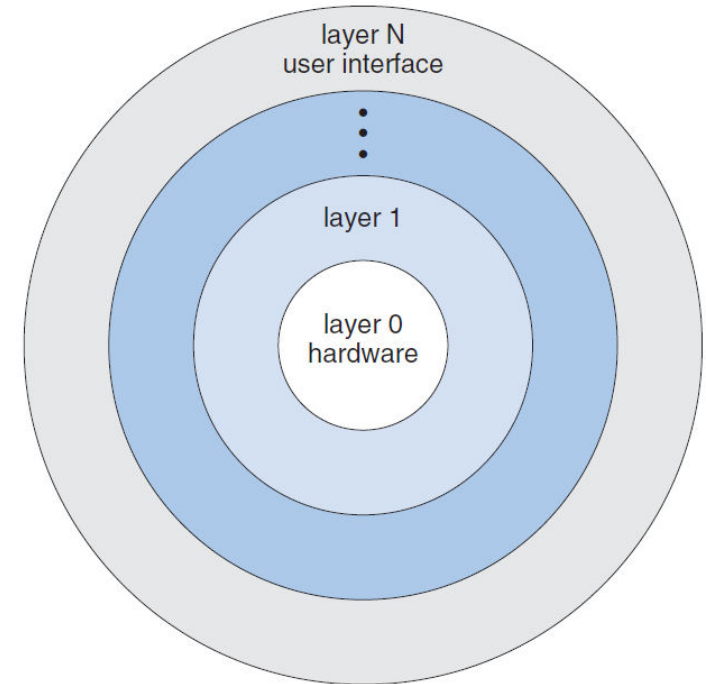


İşletim sistemi yapısı - Katmanlı Mimari

Bu modelde işletim sistemleri, iç işleyişleri değişebilen katmanlar halinde, yukarıdan aşağıya bir yapıda kararlı bileşenlerden oluşan bir yaklaşımla geliştirilmişlerdir.

Katmansal yapı ile işletim sistemleri;

- Herbir katman, sadece kendi altındaki katman tarafından verilen hizmetleri bilir,
- İlgili katman alt katmanından gelen hizmetlerin hatadan ayıklandığını kabul eder ve bağımsız geliştirilir,
- Katmanlardan birinin, herhangi bir katmanın hizmetini çağırma gereksinimi katman sıralaması açısından problemdir. Önemli olan bu sıralamaya doğru karar vermektir.
- Yüksek bir katmandan donanıma erişmek için tüm alt katmanlardan geçme gereksinimi daha düşük verimliliklere neden olabilir. Bu sorun son yıllarda daha az katmanla daha fazla fonksiyonallite yaklaşımını doğurmuştur.



İşletim sistemi yapısı – Mikro Kabuk Mimarisi

Bu yaklaşımın temeli çekirdekten olmayan servisleri kaldırmak ve bunları sistem uygulamalarıyla sunmaktır. Böylece çekirdek küçültülür.

Mikro çekirdekler temelde işleri ve belleği yönetmek ve diğer servisler arasında mesaj geçişi sağlamaktadır.

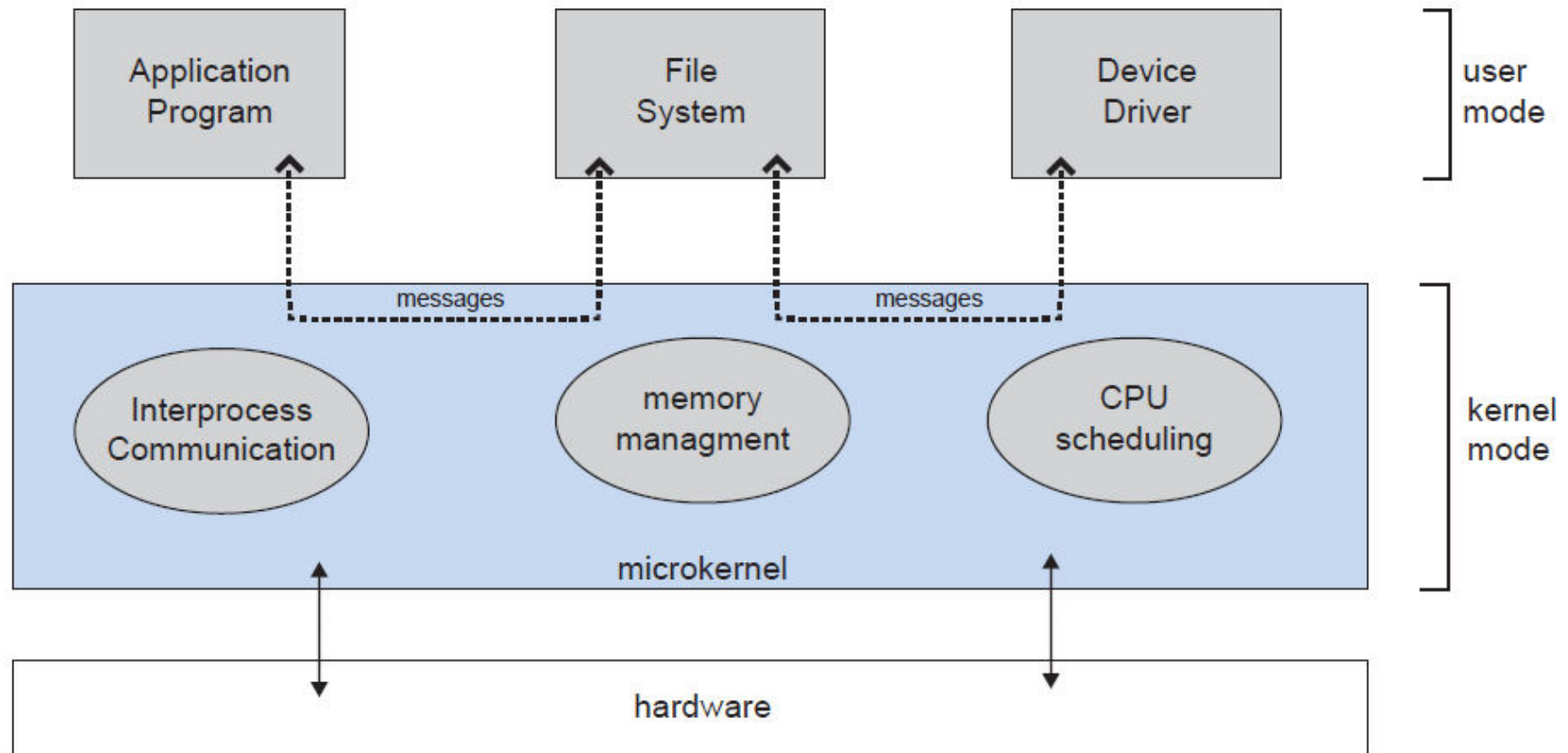
Çoğu hizmet çekirdek modundan kullanıcı moduna alındığından güvenlik ve koruma daha kolay geliştirilebilir.

Sistemin genişlemesi kolaylaşmıştır. Çünkü yeni bir sistem uygulamasının eklenmesi çekirdek derlemesini gerektirmez.

Ençok bilinen mikro çekirdek Mach (Darwin) olarak bilinen Mac OS X çekirdeğidir.

Başlangıçta Windows NT, mikro çekirdek olarak tasarlansa da XP'den sonra bu yapısı monolitik bir hal almıştır.

İşletim sistemi yapısı – Mikro Kabuk Mimarisi



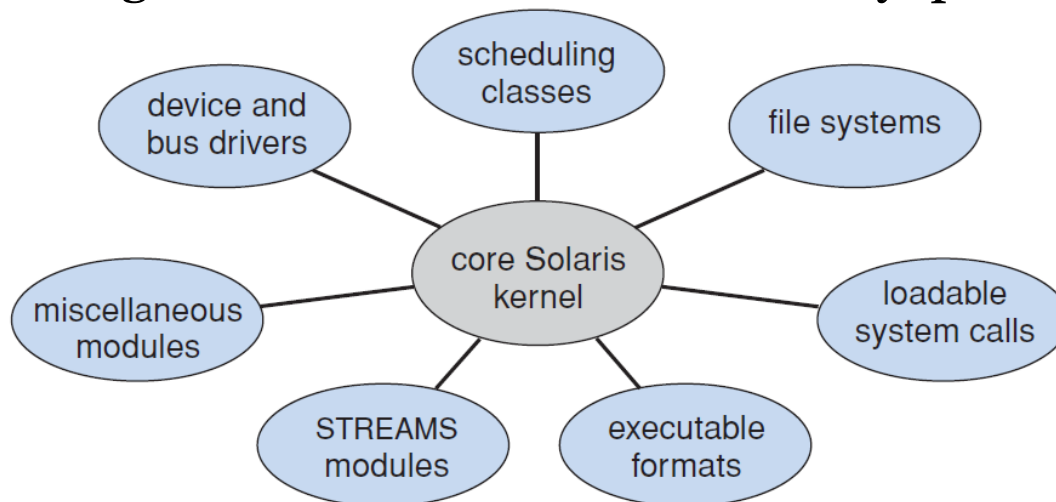
Mikro çekirdek mimarisi

İşletim sistemi yapısı - Modüler Mimari

Bu yaklaşımda, işletim sistemi, nesne yönelimli olarak, küçük bir çekirdek etrafında dinamik olarak konumlanan modüller olarak geliştirilmiştir.

Modüllerin, görevleri açıkta tanımlanmış olmaları ve arayüzleri itibariyle katmanlara benzeselerde iletişimlerinin serbest olması ve çekirdeğe doğrudan erişebilmeleri ile farklıdır.

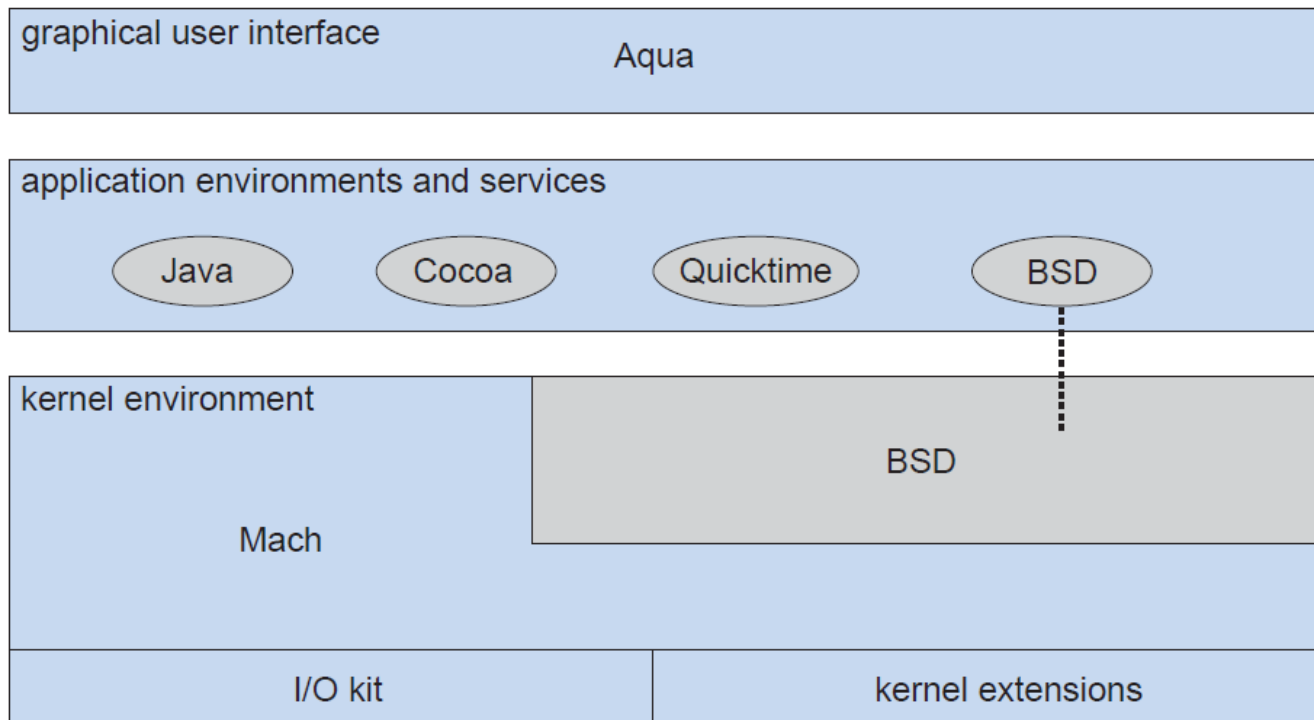
Çekirdek, mikro çekirdeğe benzer şekilde küçük ancak modüllerin birbirlerine doğrudan mesaj geçebilmelerinden dolayı mesaj geçişine sahip olmaları zorunlu değildir. Linux ve Solaris modüler yapıdadır



İşletim sistemi yapısı - Hibrit Mimari

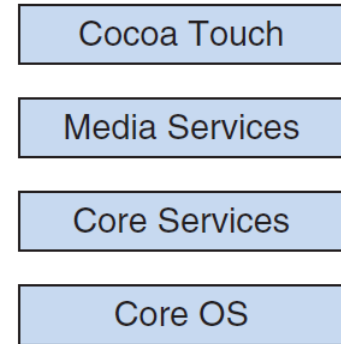
Mac OS işletim sistemleri günümüzde sabit bir mimari kullanmak yerine hibrit bir mimariyi tercih eder.

Mac OS, temelde Mach(Darwin) mikro kabuğu üzerinde BSD çekirdeğine sahip ilave servisleri barındırır. Uygulama servileri ve yüklenebilir modüller sağlayan fonksiyonlar işletim sisteminin işlevselliğini sağlar.

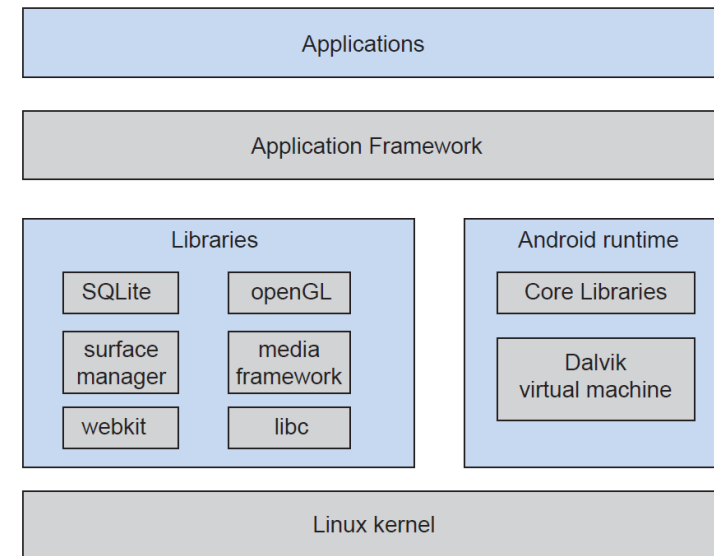


İşletim sistemi yapısı - Hibrit Mimari

iOS, Apple tarafından akıllı telefon (iPhone) ve tablet bilgisayarlar (iPad) için geliştirilmiş Mac OS X mimarisi üzerine mobil aygıtlar için ilave fonksiyonlar içeren işletim sistemidir. iOS aygıtlarında uygulama geliştirmek için farklı çatılar sunan Objective-C API'si olan **Cocoa Touch** yer alır.



Android, google öncülüğünde Open Handset Alliance kurulu tarafından akıllı telefon ve tabletler için geliştirilen ve iOS'un aksine açık kaynak kodlu mobil işletim sistemidir. Çekirdeği google tarafından geliştirilen bir Linux kabuğu kullanır.



Android yazılım geliştirme ortamı Java tabanlıdır. Ancak JVM yerine android için tasarlanan sınırlı bellek ve CPU'lara uyumlu mobil cihazlar için optimize edilmiş **Dalvik sanal makinesi** kullanılır.

İşletim sistemi - Hata ayıklama

- Hata ayıklama (**debugging**) genel olarak donanım ve yazılım olarak bir sistemdeki hataların bulunması ve bunların düzenlenmesi sürecidir.
- Bir sistemdeki sorunlar genel olarak **bug** şeklinde ifade edilir. Hata ayıklama süreci ilgili bug'ları bulma ve düzenleme sürecidir.
- Hata ayıklama, hatayı bulma ve düzeltme sürecini ifade ederken aynı zamanda performans iyileştirme/ayarlama (**performance tuning**) anlamına da gelir.
- Hata ayıklama kod yazmaktan iki kat daha zor bir süreçtir. Bu yüzden, mümkün olduğunca akıllıca kod yazarsan, hata ayıklamak için çok fazla uğraşmana gerek kalmaz. (***Kernighan Yasası***)

İşletim sistemi - Hata ayıklama

Hata Analizi

- **Debugger**'lar, işlemleri adım adım yürütmeye ve program ilerledikçe değişkenlerin ve ifadelerin incelenmesine izin verir.
- **Profiler**'lar, program yürütümü esnasında farklı bölümlerin yada kodun her satırının ne kadar zaman harcadığı yada ne kadar sistem kaynağı tükettiği gibi istatistikleri sunarlar.
- **Dumping**, yürütülen bir işlem çökerse, çökme anındaki işlemin bellek dökümü daha sonra analiz edilmek üzere diske kaydedilebilir.
- Ancak debugging sürecinde elde edilen performans ölçütleri dikkate alınmamalıdır.
- Ayrıca uygulamaların çökme nedeni diske erişememe gibi bir durumdan kaynaklanıyorsa bu durumda dump yapılamaz.

İşletim sistemi - Hata ayıklama

Performans ayarlama:

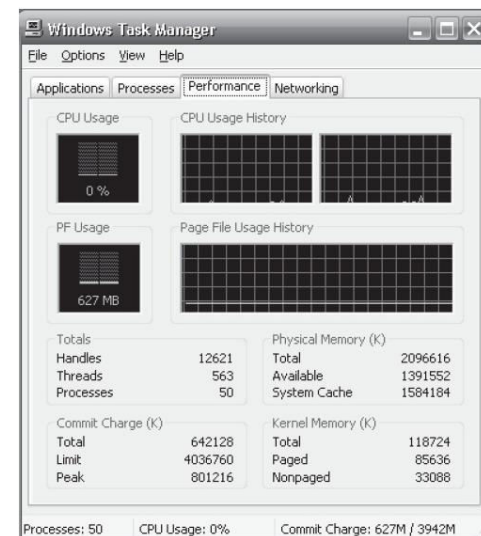
- Performans ayarlama sistem performansını izlenmesini gerektirir
- Bu yaklaşımlardan biri daha sonra analiz edilmek üzere önemli olayların saklandığı log dosyalarıdır.
- Başka bir yaklaşımsa UNIX sistemlerde «top» komutu gibi sistem durumu görüntülenebilir.
- Ayrıca windows gibi sistemlerde, sistem araçları sistem durumunu izlemenizi sağlayabilir.

Uçbirim

Dosya Düzenle Görünüm Ara Uçbirim Yardım

top - 13:55:10 up 2 min, 2 users, load average: 0,40, 0,29, 0,12
 Tasks: 126 total, 1 running, 125 sleeping, 0 stopped, 0 zombie
 %Cpu(s): 5,2 us, 4,2 sy, 0,7 ni, 88,6 id, 1,4 wa, 0,0 hi, 0,0 si, 0,0 st
 KiB Mem: 2074540 total, 349628 used, 1724912 free, 22852 buffers
 KiB Swap: 901116 total, 0 used, 901116 free, 177872 cached

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2510	root	20	0	88360	32m	4888	S	2,7	1,6	0:07.60	Xorg
3540	root	9	-11	97820	5932	4068	S	2,3	0,3	0:00.28	pulseaudio
3640	root	39	19	120m	5568	4468	S	1,7	0,3	0:00.05	tracker-extract
3622	root	20	0	80788	14m	10m	S	1,0	0,7	0:00.28	gnome-terminal
3534	root	20	0	50164	13m	11m	S	0,7	0,6	0:00.32	vmtoolsd
177	root	20	0	0	0	0	S	0,3	0,0	0:00.03	jbd2/sdal-8
2315	root	20	0	36356	3800	3168	S	0,3	0,2	0:00.85	vmtoolsd
3497	root	20	0	39224	9m	8076	S	0,3	0,5	0:00.20	metacity
3511	root	20	0	99,0m	20m	14m	S	0,3	1,0	0:01.73	gnome-panel
3537	root	20	0	63944	8268	4356	S	0,3	0,4	0:00.10	tracker-store
3544	root	20	0	99792	20m	14m	S	0,3	1,0	0:00.86	nautilus
3561	root	39	19	50176	7636	5856	S	0,3	0,4	0:00.07	tracker-miner-f
3565	root	20	0	118m	12m	9828	S	0,3	0,6	0:00.11	gnome-sound-app
1	root	20	0	2288	740	636	S	0,0	0,0	0:01.09	init
2	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.02	ksoftirqd/0
4	root	20	0	0	0	0	S	0,0	0,0	0:00.11	kworker/0:0



Kernel Modülü

- Linux kernel modülleri sistem çağrılarına erişebilen ve kernel modda çalışan programlardır,
- Özellikle donanım sürücüler ve kernel derlemesi istenmeyen yapılandırmalar kernel modülleri ile geliştirilir,
- Linux kernel modülleri ile doğrudan kernel çekirdeğine erişilebileceği için olası hata durumları, sistem güvenliği açısından göz önüne alınmalıdır,
- `lsmod` //kernel modüllerini görüntüler
- `insmod mod_ad.ko` //modül kurulumu
- `modinfo mod_ad.ko` //modül bilgisi
- `rmmmod mod_ad.ko` //modül kaldırılması
- `dmesg` //modül mesajlarının görüntülenmesi
 - `/var/log/syslog` içeriği

Kernel Modülü - Uygulama

- apt-get install build-essential linux-headers-\$(uname -r)
- apt-get install linux-source //kernel kütüphaneleri

modul_ad.c

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>
int simple_init(void) {
    printk(KERN_INFO "Modul Yuklendi\n");
    return 0;
}

void simple_exit(void) { printk(KERN_INFO "Modul Silindi\n"); }

module_init(simple_init);
module_exit(simple_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Örnek Modül");
MODULE_AUTHOR("Öğrenci");
```

Makefile

```
obj-m += modul_ad.o

all:
    make -C /lib/modules/$(shell uname -r)/build
    M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build
    M=$(PWD) clean
```

- make //module kodlarını derler
- insmod modul_ad.ko //kernel module eklendi
- dmesg | tail -1 //son kernel mesajını görüntüle
- rmmod modul_ad.ko //kernel module silindi